Panel Proposal: Automata Theory – Its Relevance to Computer Science Students and Course Contents

M. Armoni
Computer Science Dept.
The Open University of Israel,
108 Ravutski St.,
Raanana, 43107, Israel
michal@openu.ac.il

S. Rodger
Dept. of Computer Science
LSRC Rm D237, Box 90129
Duke University,
Durham, NC 27708-0129
rodger@cs.duke.edu

M. Vardi
Dept. Of Computer Science
Mail Stop 132
Rice University
Houston, TX 77005-1892
vardi@cs.rice.edu

R. Verma (moderator)
Computer Science Dept.
University of Houston
4800 Calhoun Road
Houston TX 77204-3010
rverma@uh.edu

Categories and Subject Descriptors: F.1.1 [**Computation by Abstract Devices**]: Models of Computation – *automata, computability theory, relations between models* K.3.2 [**Computers and Education**]: Computer and Information Science Education – *computer science education, curriculum*

General Terms: Languages, Theory

Keywords: automata course, relevance, curriculum

Summary

Most college and university undergraduate CS programs have a course dealing with the theory of automata and formal languages. In most institutions, the syllabus of such a course is quite stable, and if one compares the syllabus of different institutions, chances are that no significant differences will be found in the automata and computability section of the course. Since CS is a constantly evolving and rapidly developing discipline, such stability should be a matter for serious discussion and periodic re-evaluation. Moreover, the ACM/IEEE Computing Curriculum 2001 appears to downgrade this material into a unit rather than a full-fledged course. We believe that the relevance of the course is actually greater today than ever and this important issue needs intense discussions as well.

Position Statement of Dr. Michal Armoni

I believe that the main educational goal of such a course is exposing students to the thinking patterns characterizing the abstract, theoretical foundations of CS: Abstraction (e.g., through definitions of abstract models), abstract flexibility and the ability for abstract generalization (e.g., by changing or generalizing definitions of computational models, changes which are not motivated just by modeling "real-life" situations), and the ability to reason about abstract objects (e.g., studying and proving the properties of a computational model). Another educational goal is introducing students to some general important CS concepts and terms that are related to formal languages theory, such as non-determinism or computational power.

To achieve these goals, it is important to expose students to different kinds of abstract models (e.g., automata, regular expressions, grammars), with different properties (for example, some are equivalent and some are not, some share a certain closure property but differ on another). In order to emphasize non-determinism, an important recurring CS concept, but a difficult one to perceive, it is important to introduce non-deterministic variants of computational models, such that in some cases non-determinism does not increase the computational power (e.g. finite automata) while in other cases it does (e.g. pushdown automata). These considerations allow some freedom as to which computational models should be introduced. It is probably recommendable to introduce models that will be used in other courses, such as a course in compilation theory, but the important thing is not necessarily introducing as many models as possible, but rather introducing a rich enough variety, satisfying the above considerations and enabling meaningful theoretical discussions, while leaving enough time for such discussions.

Apparently, all that leads to a syllabus that is essentially very similar to the current common syllabi of such a course: quite difficult, very mathematical in nature, with no radical content changes. Indeed, abstract definitions, theorems and proofs make this a difficult and challenging course, both for students and lecturers. However, the way to cope with it is not by bypassing or softening the theoretical aspects or by focusing on other aspects, such as design within various computational models. The descending status of mathematics in the CS curriculum was the focus of many discussions. In this course, mathematics is an inherent component. It is not merely a tool, but rather the essence. I therefore believe it should remain in the center. As for the desire to change and update - this course should teach the basis of theoretical CS, and this basis did not change. The developing nature of CS should be and is expressed in other parts of our curriculum, and not necessarily in this course, which can rightfully serve as the fixed-point of the CS curriculum. This does not mean that pedagogical efforts should not be made regarding this course. However, any such effort should be aimed at finding didactic strategies that would help in conveying the contents such that the educational goals will be achieved, and not necessarily at updating these contents.

Michal Armoni is a member of the Computer Science Department at the Open University of Israel. Her research area is computer science education, currently focusing on perception of abstract CS concepts, such as reduction and non-determinism. She chaired the development team of a theoretical computational models course for CS high school students. For several years, she has been teaching the course “Computer Science Curricula” at Tel-Aviv University’s School of Education.

SIGCSE'06, March 1–5, 2006, Houston, Texas, USA.
ACM 1-59593-259-3/06/0003.

Position Statement of Dr. Susan Rodger

The formal languages and automata theory course traditionally is abstract and many students have difficulty with abstraction. We believe that the abstraction should be complemented with a hands-on problem-solving approach to experiment with the concepts and with applications. At Duke, we teach this course using the software JFLAP, a software tool for experimenting with formal languages and automata. We cover the topics of regular languages, context-free languages and recursively enumerable languages, covering both automata and grammars. Our approach and use of JFLAP is the following. With JFLAP, we build automata and grammars in all three types of languages. Students develop test sets of input strings and run their automata and grammars on them. During lecture, we use JFLAP to solve problems with the class. Sometimes we build an automaton or grammar from scratch or we illustrate a conversion such as from an NPDA to a CFG. Other times we load a file that does not correctly represent a given language, and correct it with input from students. Students use JFLAP for homework assignments, turning in their solutions electronically for grading.

We cover real world applications to aid in the understanding of the abstraction. We cover SLR parsing in which students observe the use of DFA, regular expressions, NPDA, and CFG. We cover L-systems in which students observe how scientists model the growing of plants and organisms.

Susan H. Rodger is an Associate Professor of the Practice in the Department of Computer Science at Duke University. She has been teaching the formal languages and automata theory course for sixteen years. She has developed JFLAP, www.jflap.org, software for teaching the automata theory course. She is a co-author of the book "JFLAP - An Interactive Formal Languages and Automata Package" to be published in 2006.

Position Statement of Dr. Moshe Vardi

The theory of finite automata is one of the fundamental building blocks of theoretical computer science. It is covered in numerous textbooks and in any basic undergraduate curriculum in computer science. Since its introduction in the 1950's, the theory had numerous applications in practically all branches of computer science. In these applications, finite automata are typically used in one of two roles: as models or as descriptors; finite automata are often the appropriate abstraction to model finite-state systems, and finite automata can be used as descriptors of regular languages.

Over the last two decades, I have been involved in two areas of research in which automata theory is an essential source of algorithmic tools: optimization of logic programs and specification and verification of protocols. In these applications, automata are used as a basic working tool. For example, the SPIN model checker uses nondeterministic automata as an internal representation of temporal properties. My experience has been, however, that many researchers are not comfortable working with automata theory. I believe that this is a result of the way that the theory of finite automata is typically taught.

Finite-automata theory is typically taught as a mathematical theory of computation with some applications to compiler construction. That is, finite automata are thought as a very simple and robust model of computation with applications such as lexical analysis. One rarely, however, encounters applications in a finite-automata course; concrete applications are usually left to compiler-construction courses. Thus, most students are left with the impression of finite-automata theory as an abstract mathematical theory.

I believe that the theory ought to be taught as a useful set of abstractions and tools for the working computer scientist. The educational goal should be more than just to train the students in rigorous and formal thinking; it should also be to provide the students with the knowledge of basic tools. To that end, much more emphasis should be given to applications of automata theory. I will sketch two such applications, which have found wide industrial usage. The first uses deterministic finite automata for checking equivalence of Boolean circuits, and the second uses nondeterministic automata for static checking of protocols.

Moshe Y. Vardi is the George Professor in Computational Engineering and Director of the Computer and Information Technology Institute at Rice University. He chaired the Computer Science Department at Rice University from January 1994 until June 2002. His research interests include database systems, computational-complexity theory, multi-agent systems, and design specification and verification. He has been involved in tech-transfer activities in the area of formal verification, where automata-based tool play an important role.

Position Statement of Dr. Rakesh Verma

The field of automata is vast, diverse, and growing, with string automata finding applications in lexical analysis and text editing, tree automata in logic and formal verification, and DAG automata in XML. Yet, the course gives short shrift to this diversity and to the applications of these concepts. Thus, students emerge from the course with the perception that the material is quite dated and the applications quite limited. This is most unfortunate, since the student then fails to realize that finite automata are very useful tools and must be part of the arsenal when attacking any computing problem. Consequently, the quality of software developed and verified suffers. I believe we are doing a great disservice to computer science students and to the development of the field itself consequently, by still sticking to the classical concepts of string automata and by not pointing out the diverse automata and their applications. Hence, I believe that the contents of the course need significant revision. The course contents should also be tied into the curriculum better. For example, automata-based protocol modeling/verification can be discussed in operating system and networks courses and formal verification through automata in software engineering courses. I believe that downgrading the course to a few units will only magnify the detrimental effects of the current approach.

Further, I believe that course should stay rigorous and mathematical instead of the current trend of watering down proofs and concepts. This is quite evident if one compares the older editions of texts to their latest editions. I also believe that interaction and visualization is the key to motivating and improving understanding. Thus, I am in favor of using packages such as JFLAP and RuleMaker, which is being developed at U. of Houston for visualizing tree-automata and rules in general (of which tree automata are a special case).

Rakesh Verma is a Professor of Computer Science at the University of Houston, where he has taught the Automata Theory course for over fifteen years. He and his students have developed a software interpreter for Equational Programming called LRR and a graphical user interface for LRR called RuleMaker. With RuleMaker, tree automata and rules are visualized effectively.